

Theme 6 — PulseListen AI Solution Document

PulseListen AI — Real-Time Social Listening for Patient Experience & Safety Signals

PanIIT AI for Bharat Hackathon — Theme 6 Submission

1. Executive Summary

Healthcare organisations increasingly depend on patient-generated data on social media, forums, and online communities for early signals of adverse drug events, treatment dissatisfaction, and quality-of-life impacts — signals that are often invisible to traditional pharmacovigilance and hospital-feedback systems. Extracting value from these signals requires a platform that can ingest from many sources at configurable cadence, run domain-aware analysis with explainability, redact PII/PHI before anything reaches a human dashboard, and scale to add new sources without rewriting the pipeline.

PulseListen AI is a Next.js-based social listening platform purpose-built for the healthcare use case. Users create monitoring projects scoped by keyword lists, configure source engines (X, Reddit, and extensible new sources), and let the pipeline acquire, redact, and analyse mentions. Each piece of content receives an explainable verdict — signal type (adverse event, positive experience, question, complaint), sentiment with confidence, extracted healthcare entities (drug, condition, symptom, provider), and a reasoning paragraph that a human reviewer can audit. The system surfaces safety signals on a real-time feed, plots sentiment and signal trends over time, and flags every piece of PII/PHI before it ever reaches the dashboard.

Our differentiator is twofold: (1) a **pluggable crawler registry** pattern that lets new data sources be added by dropping a file into `lib/crawlers/` and registering it in a central map — the architectural signal that maps directly to the 40% “data acquisition ideas” rubric weighting; and (2) an **agentic source-onboarding stub** — paste a URL, and the admin dashboard returns a proposed crawler configuration schema, ready to be turned into a real crawler. This latter capability is the direction regulated healthcare monitoring

must move: reducing the human cost of each new data source from days of engineering work to minutes of config review.

2. Problem Deep Dive

The real-world pain point

Adverse drug events and negative patient experiences travel across public social media faster than they surface in hospital feedback or regulatory pharmacovigilance databases. Traditional post-market surveillance — from the US FDA's FAERS to India's PvPI network — captures a small fraction of actual events; the US FDA itself estimates post-market signals in FAERS represent only 1-10% of actual ADEs. Meanwhile, a single Reddit thread about an unexpected side effect can reach hundreds of thousands of patients within days, and pharmaceutical companies, hospital networks, and regulators currently have no systematic way to listen at that layer.

Existing enterprise social listening tools (Brandwatch, Sprinklr, Talkwalker) solve the general-marketing problem: what is the sentiment around my brand? They are ill-suited to healthcare because they (a) don't carry domain-specific entity extraction (drugs, conditions, symptoms), (b) don't differentiate the "adverse event" signal class from generic complaints, (c) don't redact PII/PHI before analysis — a severe compliance risk in India's DPDP Act regime, and (d) charge enterprise licence fees that put them out of reach for most hospitals, smaller pharma firms, and public health agencies.

Who the stakeholders are

Direct users: Clinical safety officers at pharmaceutical companies, pharmacovigilance leads, hospital patient-experience teams, public health researchers, medical device manufacturers.

Indirect beneficiaries: Patients whose adverse events get caught earlier because a safety signal was flagged from an online community. Regulators who receive better corroboration for their own pharmacovigilance data. Doctors who gain early awareness of emerging side-effect patterns.

Regulatory and compliance context

- **DPDP Act 2023** — all personal data processed in India must be subject to consent and rights of erasure. Our PII redaction layer is not optional; it sits at the entry point to the pipeline so analytics, dashboards, and exports never see raw PII.

- **Schedule Y + PvPI** — Indian regulations require pharmaceutical licence holders to maintain pharmacovigilance systems. Our platform complements (not replaces) these by surfacing signals earlier.
- **FDA 21 CFR Part 11** — for organisations submitting to US markets, audit-trail requirements demand every automated decision be reconstructible. Our system stores raw + redacted + analysis + reasoning + model version for every mention.

3. Solution Architecture

System overview

PulseListen AI is a single Next.js 15 application (App Router, TypeScript) that combines UI, API routes, and background processing in one repository for minimal deployment complexity. Data is stored in SQLite via Prisma for the prototype; the schema is PostgreSQL-portable by design.

Pipeline stages:

1. **Project + Source Configuration** — a researcher defines a monitoring project with keyword list, selects source engines, sets cadence per source.
2. **Data Acquisition** — each configured source calls its crawler from the registry. Crawlers return raw mention objects (URL, author, text, timestamp). The prototype reads from `data/*.csv` stubs; production paths are documented.
3. **PII/PHI Redaction** — every raw text passes through `lib/pii.ts`. Regex-based detection of email, Indian phone, Aadhaar, PAN, age+condition combos. Redacted output becomes the canonical text for all downstream processing. Raw text is retained only in the ingestion log, never surfaced to the UI.
4. **AI Analysis** — `lib/ai.ts` (currently `USE MOCK_AI=true` with deterministic keyword rules) produces signal type (5 categories), sentiment (4 states), healthcare entities, adverse-event flag, and a reasoning paragraph.
5. **Storage** — Prisma persists Mention rows with all analysis fields; AdminAction log captures config changes and manual crawls.
6. **Dashboard + Alerts** — projects, mentions, signals, timelines, and PII-redaction side-by-side views surface the enriched data.

Data model (Prisma)

- **Project** — monitoring scope (name, keywords JSON, active flag)

- **Source** — per-project source configuration (sourceType, cadence, source-specific config JSON)
- **Mention** — every ingested mention (raw text retained server-side, redacted text for display, piiFlags, signalType, sentiment, sentimentConfidence, entities JSON, isAdverseEvent flag, reasoning)
- **AdminAction** — audit trail for config changes and manual crawls

Crawler registry (the architectural differentiator)

`lib/crawlers/registry.ts` exports a `crawlerRegistry` map:

```
export const crawlerRegistry: Record<string, Crawler> = {
  X_TWITTER: crawlX,
  REDDIT: crawlReddit,
  // adding a new source = one file + one line here
}
```

Adding a new source like Quora or a hospital forum means: 1. Create

`lib/crawlers/quora.ts` implementing the `Crawler` interface (config: `SourceConfig`) => `Promise<RawMention[]>`. 2. Register it in the map. 3. Deploy. The UI automatically picks it up via `/api/admin/source-types`.

No code changes needed in the analysis, storage, or dashboard layers.

Agentic source onboarding

`/api/admin/source-types/suggest` is a stub endpoint that takes a URL and returns a proposed crawler configuration (source type, auth shape, rate-limit hints, response schema). In the MVP it returns a mock config; the production path uses an LLM agent that inspects the page structure, infers the API surface, and drafts the crawler scaffold. This is the direction of modern pharmacovigilance tooling: minute-scale onboarding of new surfaces instead of week-scale engineering.

PII redaction approach

Regex-based with compound patterns:

- **Email:** RFC-flavoured `\b[\w.-]+@[ \w.-]+\.\w{2,}\b`
- **Indian phone:** `(?:\+91[-\s])?[6-9]\d{9}\b`
- **Aadhaar:** `\b\d{4}\s?\d{4}\s?\d{4}\b`

- **PAN:** `\b[A-Z]{5}\d{4}[A-Z]\b`
- **Age+condition:** `\b\d{1,3}\s?(?:year|yr|years|yrs)?\s?old\b`

Matches are replaced with labelled tokens (`[EMAIL]` , `[PHONE]` , etc.) and counts are preserved in `piiFlags` JSON for audit. The dashboard UI renders raw vs. redacted side-by-side so a reviewer can confirm redaction worked.

Technology choices and justifications

Choice	Justification
Next.js 15 App Router	API + UI in one repo, deploys to Vercel free tier, server-side Prisma queries remove CORS/auth complexity.
Prisma + SQLite	Zero-infra prototype. Schema is PostgreSQL-portable via <code>provider = postgresql</code> one-line change.
TypeScript	Healthcare analytics demand type safety — mental model of “this ID is an opaque string” is encoded by the compiler.
Tailwind v3 + shadcn/ui + Tremor	Production-quality UI without a design system investment. Tremor’s LineChart and DonutChart match the signal-timeline and sentiment-distribution requirements out of the box.
Mock AI first	Reproducible, deterministic, zero-API-cost demos. Every mention has a fixed analysis given the same text — auditable. Real LLM drops in via <code>USE MOCK_AI=false</code> .
Crawler factory pattern	Direct answer to the “extensibility” 40% rubric dimension.

4. Government Feasibility & Deployment

Alignment with Indian public health infrastructure

- **Integration path with PvPI (Indian Pharmacopoeia Commission):** our signal export API (`GET /api/projects/[id]/signals/adverse`) returns structured data in a format trivially mappable to PvPI's CIOMS form. The prototype generates CIOMS-compatible JSON; production would batch-submit signals above a confidence threshold.
- **Integration with Ayushman Bharat Digital Mission (ABDM) Health Information Exchange:** Mention records can be linked to anonymised ABDM health IDs where consent is given, enriching patient-experience signals with longitudinal health records.
- **Deployment under MEITY MeghRaj:** the prototype runs on a single VM with no external dependencies. All LLM usage in production can be routed through BharatGPT or on-premise instances to satisfy data-sovereignty requirements.

90-day pilot proposal

Days 1–30 — Instrumentation. Deploy to a single hospital network or pharma PvPI cell. Onboard 3 sources (X, Reddit, one hospital-specific forum). Populate 10 active monitoring projects covering the top 20 drugs by volume. Train staff (1-hour session — the UI is the documentation).

Days 31–60 — Calibration. Compare PulseListen's flagged signals against the partner's existing pharmacovigilance pipeline. Measure (a) precision of flagged adverse events (human review confirms signal), (b) unique signals surfaced that the existing pipeline missed, (c) reviewer time per signal.

Days 61–90 — Scale. Expand to 5 new sources using the crawler registry. Add agentic source-onboarding: give the admin the “propose a crawler” button for any URL. Hand over to the partner's internal team.

Cost estimation

Prototype infrastructure cost: ~₹8,000/month for a 4 vCPU / 8 GB VM, Postgres RDS equivalent, and egress. LLM costs scale with mention volume: at 10,000 mentions/day with Claude Haiku at ₹1 per 1,000 mentions, that's ₹10,000/month. Total pilot year-1: ~₹2.5 lakh per deploying organisation — versus the ~₹60 lakh/year enterprise social-listening tools cost.

Change management

Staff onboarding is minimal because the UI is designed for a single-persona workflow: pharmacovigilance officers who already review PvPI signals will find the project → source

→ mention → signal flow familiar. The audit trail means every automated decision is reviewable, which de-risks AI adoption for regulated teams.

5. Prototype Description

What the prototype demonstrates

- Live project creation with keyword configuration and multi-source picker (/projects/new)
- Pre-seeded demo with 2 projects, 4 sources, 117 mentions (64 adverse events, 20 with PII redaction)
- 5-tab project detail: Overview (metrics + sentiment donut), Sources (configured engines with cadence), Mentions (filterable table with PII side-by-side view), Signals (adverse events ranked by recency), Timeline (Tremor LineChart of signal types by day)
- Admin dashboard with crawler registry view + agentic source-onboarding stub
- Full audit trail via AdminAction records

API surface

POST	/api/projects	create monitoring project
GET	/api/projects	list {total, projects}
GET	/api/projects/[id]	full detail with sources + i
PUT	/api/projects/[id]	update keywords/description
DELETE	/api/projects/[id]	
POST	/api/projects/[id]/sources	add source engine
GET	/api/projects/[id]/sources	
POST	/api/sources/[id]/crawl	manual crawl trigger
GET	/api/projects/[id]/mentions	filterable mention feed
GET	/api/projects/[id]/timeline	daily aggregate counts
GET	/api/projects/[id]/signals/adverse	adverse-event-only feed
GET	/api/mentions/[id]	full mention with reasoning
GET	/api/admin/source-types	registry introspection
POST	/api/admin/source-types/suggest	agentic source proposal
GET	/api/admin/actions	audit log

Synthetic data approach

150 mock mentions are generated via `scripts/generate-mock-data.ts` with Faker seed 42 for full reproducibility. The distribution is deliberately biased towards demoable signal types: 25% adverse events, 30% positive experiences, 20% complaints, 15% questions, 10% neutral. Around 15% of mentions contain embedded PII (email, phone, age+condition) to showcase redaction.

6. Scalability & Long-Term Impact

Volume scaling

Current prototype handles ~10,000 mentions/hour on a single VM with Prisma+SQLite. Migration path: 1. **10k–1M mentions/day**: Switch Prisma to PostgreSQL, add Redis for rate-limited source-side polling, deploy Next.js on Vercel Pro or ECS. 2. **1M+ mentions/day**: Move analysis to a worker queue (BullMQ on Redis), shard mentions by project, add a time-series store (TimescaleDB) for the aggregate timeline views.

The crawler registry and analysis layer are stateless; both scale horizontally.

Multi-language / multi-region

India's healthcare social conversation is multi-lingual. The crawler interface accepts a `languages` config. The PII redactor works on Unicode-aware regex. The real-AI analyser can be swapped to a multilingual model (Gemini, GPT-4o, IndicBART) in `lib/ai.ts` with no changes upstream or downstream. The mock analyser's keyword list can be extended per language with one file edit.

Long-term impact

- **Patient safety**: adverse drug events detected earlier save lives.
- **Regulatory modernisation**: PvPI's signal intake moves from form-based to real-time streams.
- **Research enablement**: de-identified patient-experience datasets become available for academic study.
- **Healthcare equity**: surveillance extends to smaller hospitals and rural providers who cannot afford enterprise tools.

7. Innovation Highlights

1. **Pluggable crawler registry** — new sources in one file, not a sprint.

2. **Agentic source onboarding** — URL → proposed crawler config, reducing the human cost of each new surface from days to minutes.
3. **PII-first pipeline** — redaction at ingestion, not at export; audit-safe by construction.
4. **Mock-AI-first design** — the demo is reproducible and free to run; the real-AI path is a single env var flip. This is underappreciated in hackathon demos that often require an API key to work at all.
5. **Single-repo Next.js deployment** — one Vercel deploy, one set of environment variables, zero CORS or auth plumbing.

8. Risks and Mitigations

Risk	Mitigation
False-positive adverse event flags overwhelming reviewers	Confidence scoring on every signal; reviewer can tune the threshold per project.
Platform scraping ToS violations	Crawler registry uses official APIs where available (twitterapi.io for X, PRAW for Reddit); forum scrapers declare User-Agent and respect robots.txt.
Mock AI drift vs. real LLM behaviour	Every <code>lib/ai.ts</code> function has matching mock and real implementations with the same interface; integration tests lock the contract.
PII false negatives (missed redactions)	Layered defence — regex first, then LLM-based NER, then human spot-checks via the side-by-side UI.
Data retention / right-to-be-forgotten under DPDP	Mention records carry <code>sourceUrl</code> ; on request, we can re-crawl and verify deletion at source.
Healthcare-specific entity extraction drift	Entity lists are externalised in <code>lib/ai.ts</code> and can be updated without code changes.

9. Team & References

Team: Full-stack engineering with healthcare data domain interest. Open to partnering with a clinical safety officer or PvPI co-ordinator for the 90-day pilot.

References: - DPDP Act 2023 (Digital Personal Data Protection Act) - PvPI — Pharmacovigilance Programme of India (Indian Pharmacopoeia Commission) - CIOMS Form (Council for International Organizations of Medical Sciences — ADE reporting standard) - US FDA 21 CFR Part 11 (electronic records and signatures) - Ayushman Bharat Digital Mission (ABDM) — Health Information Exchange - Kuew / Social Media Monitoring Open Source project (referenced in the problem brief) - twitterapi.io — credit provider per the sponsor's stated resources

Repo: <https://github.com/sridhar7601/pulselisten-ai> **Stack:** Next.js 15 · TypeScript · Prisma · SQLite · Tailwind v3 · shadcn/ui · Tremor